The Journal of Open Source Software

CNearest: A c++ toolkit to use the nearest neighbour algorithm to solve fluid problems

Neil A. Butcher[1*], **James Tyrrell**[1*], **and David Smith**[2*]

1 ARC, University of Birmingham, Edgbaston, Birmingham, UK **2** School of Mathematics and Centre for Systems Modelling and Quantitative Biomedicine, University of Birmingham, Edgbaston, Birmingham, UK ***** These authors contributed equally.

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

This package implements a method for solving problems in very viscous fluid dynamics. The package provides a method for calculating the drag force and moment on rigid spheres and spheroids immersed in a 'simple' (Newtonian) viscous fluid, in the limit where inertia is negligible relative to viscosity (zero Reynolds number / Stokes flow). This system is particularly relevant in microscale biological fluid dynamics systems and colloids; one specific application is the calculation of the diffusion tensor of a specified structure. The underlying numerical algorithm is the nearest-neighbour implementation (Gallagher et al., 2019; Smith, 2018) of regularised stokeslets (Cortez, 2001; Cortez et al., 2005); this approach enables efficient and accurate calculations requiring only surface point discretisations rather than surface or volumetric meshes and completely avoids. The calculated flow field is also ensured to be regular throughout the interior, exterior and surface of the body. It is a feature-complete reimplementation and further development of the key test cases in Smith (2018). The package also serves as a library that can be utilised with the user's own surface discretisation to calculate the force and moment on any given three dimensional body undergoing rigid body motion. The library also forms an extensible toolkit for problems involving calculations of flow fields due to flexible motions, such as cilia-driven transport, motility / swimming due to flagellar movement or surface deformations, and integration with models of actively bending elastic bodies.

Statement of need

Microscale biological flow problems typically involve complex-shaped bodies undergoing large deformations, which renders analytical calculations or methods based on volumetric meshing (finite element, finite difference) challenging to implement. However, in the case where the fluid has Newtonian or approximately Newtonian properties, following non-dimensionalisation the flow can be modelled through the linear Stokes flow equations,

$$-\boldsymbol{\nabla} p + \nabla^2 \boldsymbol{u} = 0, \quad \nabla \cdot \boldsymbol{u} = 0, \quad (1)$$

where $p(\boldsymbol{x})$ and $\boldsymbol{u}(\boldsymbol{x})$ are respectively pressure and velocity at the point $\boldsymbol{x}$.

One of the basic problems of Stokes flow is the calculation of the force and moment on a body immersed in a fluid and subject to no-slip, no-penetration boundary conditions, due to specific rigid body motion. Mathematically, for body with surface B subject to translational velocity $\boldsymbol{U}$ and angular velocity $\boldsymbol{\Omega}$ about the origin, the problem is to solve equation (1) subject to the boundary condition,

$$\boldsymbol{u}(\boldsymbol{x}) = \boldsymbol{U} + \boldsymbol{\Omega} \times \boldsymbol{x}, \quad \text{for all} \quad \boldsymbol{x} \in B, \quad (2)$$

DRAFT

and thereby calculate the force and moment

$$\boldsymbol{F} = \iint_B \boldsymbol{\sigma} \cdot \boldsymbol{n} dS(\boldsymbol{x}), \quad \boldsymbol{M} = \iint_B \boldsymbol{x} \times \boldsymbol{\sigma} \cdot \boldsymbol{n} dS(\boldsymbol{x}), \quad (3)$$

where the stress tensor is given in terms of the pressure and velocity by,

$$\boldsymbol{\sigma} = -p\boldsymbol{I} + \boldsymbol{\nabla u} + (\boldsymbol{\nabla u})^T, (4)$$

is the stress tensor. The system (1-4) forms the basis for more complex problems in which the body motion is not prescribed, or the surface motion is non-rigid. Moreover due to the linearity and instantaneity of the Stokes flow equations, the solution for translational velocity (U) and angular velocity (Ω) can be determined from a linear combination of the 'basic' modes corresponding to unit velocity along each cartesian axis and unit angular velocity about each cartesian axis; providing these calculations forms the basic functionality for this package. The (6×6) matrix of forces and moments corresponding to each basic rigid body model is referred to as the *grand resistance matrix* (Pozrikidis, 2002) or *frictional tensor* (Wegener, 1981). The inverse of this matrix can be used to calculate the diffusion tensor quantifying the translational and rotational Brownian motion of the body (Wegener, 1981).

Linearity means that (1-4) can be solved through techniques such as the boundary element method and method of fundamental solutions, which result in a linear algebra problem of the form,

$$Af = u,$$

where A is a $3N \times 3N$ matrix formed by evaluating or integrating fundamental solutions of (1), f is a $3N$-vector of unknown traction or force values at each degree of freedom, and u is a $3N$-vector of velocities at each point of the discretisation. For more details, see for example references (Pozrikidis, 1992, 2002). However these techniques involve singular solutions which present challenges for numerical implementation. Moreover, despite its high numerical efficiency, the boundary element method requires the user to supply a true surface mesh with connectivity table and local basis system. The method of regularised stokeslets (Cortez, 2001; Cortez et al., 2005) has achieved very significant success in biological fluid dynamics (see for example the Special Issue edited by Cortez and Olson (Cortez & Olson, 2021)) by removing both the mathematically singular behaviour, and avoiding the need to construct a true mesh (see also Smith (2009)).

The method of regularised stokeslets solves the resistance problem by forming a $(3Q \times 3Q)$ matrix of stokeslet evaluations between each combination of (Q) surface points (each stokeslet is a (3×3) rank 2 tensor), then inverting this matrix to determine stokeslet strengths (corresponding to surface forces) from the prescribed surface velocity. These stokeslet strengths can then be summed to calculate total force and moment on the body (corresponding to equation (3)), and moreover used to evaluate the velocity at any point in the fluid through weighting a regularised stokeslet sum. The mobility problem is a modest generalisation which involves instead prescribing the total force and moment and inverting a $(3(Q+2) \times 3(Q+2))$ matrix for the stokeslet strengths and rigid body motion (U), (Ω). More advanced extensions such as the swimming problem involve extending the mobility problem by prescribing a swimming stroke relative to the local frame (Gallagher & Smith, 2018), or integrating with an active elastohydrodynamic model (Hall-McNair et al., 2019). We will not recapitulate the mathematical definition of the regularised stokeslet in the present paper but instead refer the reader to reference (Cortez et al., 2005). Physically, the regularised stokeslet can be thought of as the flow and pressure fields resulting from a unit force concentrated on a small but finite region of fluid, and the method of regularised stokeslets approximates the flow around a body by a weighted sum of these forces.

The method of regularised stokeslets is subject to three primary sources of numerical error: (1) regularisation error, (2) discretisation error associated with the surface force distribution, (3) discretisation error associated with the approximation of the surface integral (Cortez et al.,

DRAFT

2005; Gallagher et al., 2019). Sources (1) and (3) have an opposite dependence on the small regularisation (ϵ), the effect of which is that attempting to reduce the regularisation parameter by taking ($\epsilon \to 0$) leads to a requirement for increasingly refined surface discretisations. Because the number of degrees of freedom is proportional to the number of discretisation points, the effect is that an increasingly large dense matrix must be inverted, resulting in high computational cost. One strategy to address this issue that has been developed is the use of two surface discretisations: a coarse force discretisation with (N) points - which determines the number of degrees of freedom, and a finer quadrature discretisation with (Q) points - which requires more kernel evaluations but does not affect the size of the matrix to be inverted. Projecting the force to the quadrature discretisation through nearest-neighbour interpolation leads to the 'NEAREST' method (Smith 2018) which achieves very significant improvements in accuracy and efficiency with fairly minimal additional complexity of implementation. The NEAREST method has been applied to model the diffusion tensor of a novel colloidal construct (Smith, 2018), the motility of groups of swimming cells (Gallagher & Smith, 2018, 2020), revealing the propulsive role of the sperm flagellar endpiece (Neal et al., 2020) and calculating flow in a multiciliated organ (Gallagher & Smith, 2020).

Despite the proven effectiveness of the method, a barrier to wider adoption of NEAREST is that the existing implementation underpinning most of the references above is an undocumented research code implemented in Matlab. That implementation primarily uses built-in linear algebra operations throughout the workfrom, from stokeslet evaluation through matrix assembly, and avoids the use of for loops. While this approach has some advantages - notably brief code and very simple exploitation of parallelism (Gallagher & Smith, 2020), it tends to result in the storage of large stokeslet matrices, which can be highly inefficient as the quadrature discretisation is refined. Along with the lack of documentation and an extensible framework, the NEAREST code base has not yet been widely adopted, despite its demonstrated capabilities on problems of widespread interest in microscale biological flow.

cNEAREST is therefore designed to address the need for an extensible, accessible and efficient methodology for Stokes flow problems. To enable efficient and scalable use of computing resources, the package is implemented in C++, with the widely-used *eigen* package handling linear algebra operations (G. et al., 2010). The package also implements three specific algorithmic developments, which arose through the process of re-implementing the method in C++:

1. Tiebreaking
2. Stokeslet accumulation
3. Rescaling by local stokeslet count

These changes are described in more detail in the reimplementation section.

Usage

The workflow is that the user will first define the problem in terms of physical bodies such as spheres or spheroids (implementation of other shapes and flexible motions is an area for future development) from which the force and quadrature discretisations are created, then finally defining the boundary conditions. For the rigid body resistance problem (1-4) with no-slip, no-penetration boundary conditions, the boundary velocity can be calculated from supplied values of the translational and rotational velocity of the body, from which the surface velocity components at each point are calculated.

The library then constructs a linear algebra problem by detecting nearest neighbours, managing degrees of freedom, assembling matrices from the regularised stokeslets, and assembling vectors from the boundary conditions. A linear solver (either the default solver based on LU-decomposition with pivoting or alternative chosen by the user) is invoked to find the unknown force distribution over the nodes. Finally the same tools which constructed the full

DRAFT

size vectors are used to decompose them back to the total physical force and moment on the body.

Validation

The results of the code have been compared with the results of the (established) Matlab codebase. This comparison goes to make one of our suites of unit tests. This code is also capable of reproducing the results tables published in Smith (2018) for the force and moment on a translating and rotating sphere and prolate spheroid. This comparison is presented as example 1 in how to use our code (** where does this refer to?**).

Reimplementation in C++

Reimplenting the functionality previously in Matlab (Gallagher, 2018) into C++ was not solely a language change but also afforded an opportunity to make the code more reusable and efficient. The library is designed to be adaptable to different problems by making a general interface for discretisations, therefore even if our own library does not include a discretisation for the shape a consumer needs, one can be written outside of the library and injected in at runtime. The library is also designed to be adaptable to future innovations by trying to predict potential areas for development (such as the linear solver, and the algorithm for splitting between tied neighbours), at these points in the code we have injected the dependency in the form of an interface, rather than hard code the options inside the library.

Reimplementation also suggested several substantive developments in the nearest neighbour algorithm that have been incorporated into cNEAREST.

1. *Tiebreaking* refers to the situation in which a quadrature point is equidistant between force points to within double precision floating point error. In this situation, different computer programs, and potentially even different binaries for the same program, may produce different mappings and hence slightly different solutions. cNEAREST provides the option of sharing a stokeslet contribution between two or more force points, and does so smoothly, to minimise this sensitivity.

2. *Stokeslet accumulation* refers to the method of assembling the left hand side matrix from stokeslet sums. Motivated by the aim of avoiding for-loops, the original NEAREST implementation created a $(3N \times 3Q)$ stokeslet matrix then multiplied by a $(3Q \times 3N)$ sparse matrix to produce the $(3N \times 3N)$ left hand side. As (Q) grows, the $(3N \times 3Q)$ double precision values occupies sufficient memory to significantly slow the computation. By contrast, the stokeslet accumulation option in cNEAREST takes the approach of adding the the (3×3) stokeslet values into the $(3N \times 3N)$ matrix one-at-a-time through a for-loop.

3. *Rescaling by local stokeslet count* concerns the issue that the original NEAREST implementation worked by summing stokeslets, so that each degree of freedom corresponded to the 'share' of the local force ascribed to each quadrature point. The effect of this choice was that as the number of quadrature points is increased, the magnitude of the left hand side matrix increases without bound, complicating convergence tests on the left hand side itself. Instead, cNEAREST provides the option to rescale by the number of quadrature points local to each force point, so that the left hand side matrix converges as (Q) is increased, and each degree of freedom corresponds simply to the force associated with each force point.

DRAFT

Summary and areas for future development

Potential areas for future development include solution of model cells with flexible flagella, swimming problems (mobility due to a prescribed swimming stroke; (Gallagher & Smith, 2018)), multiple bodies and parallelisation (Gallagher & Smith, 2020), boundary image systems (Ainley et al., 2008; Cortez & Varela, 2015), stokeslet lines Cortez (2018), stokeslets with improved far field behaviour (Zhao et al., 2019), integration with elastohydrodynamics (Hall-McNair et al., 2019), stokeslet rings (Tyrrell et al., 2019), Richardson extrapolation (Gallagher & Smith, 2021), implementation of the double layer potential, which is highly important for problems involving surface slip (Smith et al., 2021), modelling self-motile phoretic colloids (Montenegro-Johnson et al., 2015) and fast multipole acceleration for very large problems (Rostami & Olson, 2016).

Acknowledgements

This work was partially supported by the Engineering and Physical Sciences Research Council (EP/N021096/1).

We acknowledge contributions from Simon Hartley, whose earlier engagement with this problem uncovered the risk of force points which were tied in for nearest neighbour.

References

Ainley, J., Durkin, S., Embid, R., Boindala, P., & Cortez, R. (2008). The method of images for regularized Stokeslets. *J. Comput. Phys.*, *227*(9), 4600–4616.

Cortez, R. (2001). The method of regularized Stokeslets. *SIAM J. Sci. Comput.*, *23*(4), 1204–1225.

Cortez, R. (2018). Regularized stokeslet segments. *J. Comput. Phys.*, *375*, 783–796.

Cortez, R., Fauci, L., & Medovikov, A. (2005). The method of regularized Stokeslets in three dimensions: Analysis, validation, and application to helical swimming. *Phys. Fluids*, *17*(3).

Cortez, R., & Olson, S. (2021). *20 years of regularized Stokeslets: Applications, computation and theory*.

Cortez, R., & Varela, D. (2015). A general system of images for regularized Stokeslets and other elements near a plane wall. *J. Comput. Phys.*, *285*, 41–54.

G., G., J., B., & others. (2010). *Eigen v3*. http://eigen.tuxfamily.org.

Gallagher, M. T. (2018). NEAREST. In *gitlab repository*. gitlab. https://gitlab.com/meuriggallagher/NEAREST

Gallagher, M. T., Choudhuri, D., & Smith, D. J. (2019). Sharp quadrature error bounds for the nearest-neighbor discretization of the regularized stokeslet boundary integral equation. *SIAM J. Sci. Comput.*, *41*(1), B139–B152.

Gallagher, M. T., & Smith, D. J. (2018). Meshfree and efficient modeling of swimming cells. *Phys. Rev. Fluids*, *3*(5), 053101.

Gallagher, M. T., & Smith, D. J. (2020). Passively parallel regularized stokeslets. *Phil. Trans. R. Soc. A*, *378*(2179), 20190528.

Gallagher, M. T., & Smith, D. J. (2021). The art of coarse Stokes: Richardson extrapolation improves the accuracy and efficiency of the method of regularized stokeslets. *R. Soc. Open Sci.*, *8*(5), 210108.

Hall-McNair, A. L., Montenegro-Johnson, T. D., Gadêlha, H., Smith, D. J., & Gallagher, M.

DRAFT

T. (2019). Efficient implementation of elastohydrodynamics via integral operators. *Phys. Rev. Fluids*, *4*(11), 113101.

Montenegro-Johnson, T. D., Michelin, S., & Lauga, E. (2015). A regularised singularity approach to phoretic problems. *Eur. Phys. J. E*, *38*, 1–7.

Neal, C. V., Hall-McNair, A. L., Kirkman-Brown, J., Smith, D. J., & Gallagher, M. T. (2020). Doing more with less: The flagellar end piece enhances the propulsive effectiveness of human spermatozoa. *Phys. Rev. Fluids*, *5*(7), 073101.

Pozrikidis, C. (1992). *Boundary integral and singularity methods for linearized viscous flow*. Cambridge University.

Pozrikidis, C. (2002). *A practical guide to boundary element methods with the software library BEMLIB*. CRC.

Rostami, M. W., & Olson, S. D. (2016). Kernel-independent fast multipole method within the framework of regularized stokeslets. *J. Fluids Struct.*, *67*, 60–84.

Smith, D. J. (2009). A boundary element regularized Stokeslet method applied to cilia-and flagella-driven flow. *Proc. R. Soc. Lond. A*, *465*(2112), 3605–3626.

Smith, D. J. (2018). A nearest-neighbour discretisation of the regularized stokeslet boundary integral equation. *J. Comput. Phys.*, *358*, 88–102. https://doi.org/10.1016/j.jcp.2017.12.008

Smith, D. J., Gallagher, M. T., Schuech, R., & Montenegro-Johnson, T. D. (2021). The role of the double-layer potential in regularised stokeslet models of self-propulsion. *Fluids*, *6*(11), 411.

Tyrrell, J., Smith, D. J., & Dyson, R. J. (2019). Regularized stokeslet rings: An efficient method for axisymmetric Stokes flow with application to the growing pollen tube. *Phys. Rev. Fluids*, *4*(6), 063102.

Wegener, W. A. (1981). Diffusion coefficients for rigid macromolecules with irregular shapes that allow rotational-translational coupling. *Biopolymers*, *20*(2), 303–326. https://doi.org/https://doi.org/10.1002/bip.1981.360200205

Zhao, B., Lauga, E., & Koens, L. (2019). Method of regularized stokeslets: Flow analysis and improvement of convergence. *Phys. Rev. Fluids*, *4*(8), 084104.

DRAFT